

Lab 4: Speaker and Tones

Objectives

- Use the M5Stack built-in speaker to produce tones using `M5.Speaker.tone()`.
- Control speaker volume with `M5.Speaker.setVolume()`.
- Store a sequence of musical notes in an array and iterate over it.
- Write and call a void function to organize repeated code.
- Distinguish between blocking and non-blocking playback and choose the right approach for each situation.
- Implement play/pause control by managing state with a `boolean` variable and a current-note index.
- Understand the concept of tones (notes) and a speaker
- Practice more complex sketches (switch-case, loop)

Key Concepts:

1. The M5Stack Speaker

The M5Stack has a small built-in speaker. You can make it produce a tone by telling it two things: the frequency of the sound (how high or low the pitch is, measured in Hz) and how long to play it (in milliseconds).

Function	Description
<code>M5.Speaker.setVolume(v)</code>	Set volume. Range: 0 (silent) to 255 (loudest). Recommended: 128.
<code>M5.Speaker.tone(freq, duration)</code>	Play a tone at frequency <code>freq</code> (Hz) for duration milliseconds. Non-blocking – <code>loop()</code> continues immediately
<code>M5.Speaker.tone(freq)</code>	Play a tone at frequency <code>freq</code> until <code>stop()</code> is called. Useful when the user controls how long the note plays.
<code>M5.Speaker.stop()</code>	Stop the speaker immediately.

Setting the volume — call this once in `setup()`:

```
M5.Speaker.setVolume(128); // range 0 (silent) to 255 (loudest)
```

Playing a tone

```
// Play a tone at frequency freq for duration milliseconds
M5.Speaker.tone(freq, duration);

// Stop the speaker immediately
M5.Speaker.stop();
```

Important: `tone()` is non-blocking. This means it starts the sound and returns immediately — `loop()` keeps running while the sound plays. You will explore what this means in Task 1.

2. Musical Notes and Frequencies

Every musical note corresponds to a specific frequency. The table below shows the notes in octave 4, which is the middle range of a piano.

Note	Exact Frequency (Hz)	Frequency Used (Hz)	Define
C3 (Do)	130.81	131	<code>#define C3 131</code>
C4 (Do)	261.63	262	<code>#define C4 262</code>
D4 (Re)	293.67	294	<code>#define D4 294</code>
E4 (Mi)	329.63	330	<code>#define E4 330</code>
F4 (Fa)	349.23	349	<code>#define F4 349</code>
G4 (Sol)	392.00	392	<code>#define G4 392</code>
A4 (La)	440.00	440	<code>#define A4 440</code>
B4 (Si)	493.89	494	<code>#define B4 494</code>
C5 (Do)	523.26	524	<code>#define C5 524</code>

Use `#define` to give each frequency a readable name. This is like creating a constant shortcut: every time the compiler sees `C4` it replaces it with `262`.

```
#define C4 262    // Middle C – 262 Hz
#define D4 294
```

3. Arrays

An array is a collection of values of the same data type stored under one variable name – like a row of labelled boxes, each holding one value. Every box has a number called an index, starting at 0.

Declaring and initializing:

```
int melody[] = { G4, G4, G4, D4 }; // 4 notes in a row
```

Accessing elements:

Access individual notes by index:

```
melody[0] is G4
melody[1] is G4
melody[2] is G4
melody[3] is D4
```

Counting elements automatically:

```
int numNotes = sizeof(melody) / sizeof(melody[0]);
// sizeof(melody) -> total bytes used by the whole array
// sizeof(melody[0]) -> bytes used by one element
```

When you change `melody` array, `numNotes` is automatically calculated.

4. The for Loop

A for loop repeats a block of code a set number of times. It is ideal for stepping through every element in an array.

```
for (int i = 0; i < numNotes; i++) {  
    // i starts at 0, increases by 1 each round  
    // the loop runs while i < numNotes  
    M5.Speaker.tone(melody[i], 500);  
    delay(500);  
}
```

Think of it like a to-do list: start at item 0, do it, cross it off, move to item 1, and so on until the list is finished.

5. Calculating Play Time from Tempo

In music, tempo is measured in BPM (beats per minute). A note value of 1 means one beat. To convert a note to milliseconds:

```
// 60000 ms in one minute / tempo BPM = ms per beat  
int playTime = noteValue * (60000 / tempo);  
  
// Example: noteValue = 1, tempo = 120 BPM  
//   playTime = 1 * (60000 / 120) = 500 ms  
  
// Example: noteValue = 2 (half note), tempo = 120 BPM  
//   playTime = 2 * (60000 / 120) = 1000 ms
```

6. Functions

A function is a named block of code that you can call by name. You have already used functions such as `setup()`, `loop()`, and `M5.Display.println()`. In this lab you will write your own.

When the same code appears in several places, put it in a function. This makes your code shorter, easier to read, and easier to fix — you only need to change one place.

```
// Function definition — written below loop()  
void showInfo(int currentTempo) {  
    M5.Display.clear();  
    M5.Display.setCursor(0, 0);  
    M5.Display.print("BPM: ");  
    M5.Display.println(currentTempo);  
}
```

Once function is defined, you can call the function from other functions like, `setup()` or `loop()`.

```
// Calling the function from setup() or loop()  
showInfo(120);
```

The keyword `void` means the function does not return a value. The word in parentheses (`int currentTempo`) is a parameter — a value passed in when you call the function.

Note: Arduino automatically generates function prototypes, so you can define functions below `loop()` and still call them from `setup()` and `loop()` without any extra declarations.

7. Blocking vs Non-Blocking Playback

Because `tone()` is non-blocking, you must decide when to wait and when to let `loop()` continue.

Approach	Code	Effect
Blocking (waits)	<code>M5.Speaker.tone(C4, 500);</code> <code>Delay(500);</code>	<code>loop()</code> is frozen for 500 ms. No Buttons can be detected while waiting.
Non-blocking (continues)	<code>M5.Speaker.tone(C4, 500);</code> <code>// no delay</code>	<code>loop()</code> keeps running. Buttons still work, but tones from different buttons can overlap.

To start a new tone immediately and cleanly, call `stop()` first:

```
M5.Speaker.stop();
M5.Speaker.tone(newFreq, duration); // plays cleanly, no overlap
```

Always use `wasPressed()` with the speaker — never `isPressed()`. If `isPressed()` calls `tone()` hundreds of times per second, the speaker adds each tone on top of the previous one, producing an unexpectedly loud or distorted sound.

8. Responsive Playback with `currentIndex`

In Tasks 2 and 3 the melody plays inside a `for` loop — the entire melody runs before `loop()` can check buttons again.

In Task 4 we switch to a different approach: instead of a `for` loop, we keep track of the current note in a global variable `currentIndex` and play **one note per `loop()` cycle**. After each note, `loop()` checks the buttons before deciding whether to continue.

- **for loop approach:** musician plays the whole song without stopping — you cannot pause mid-song.
- **`currentIndex` approach:** musician plays one note, then asks the conductor "shall I continue?" — you can pause at any moment.

Example Code

The sketch below demonstrates the core speaker functions. Upload it and listen to the three tones.

```
#include <M5Unified.h>
#define C4 262 // Define note frequencies as named constants
#define E4 330
#define G4 392

void setup() {
  M5.begin();
  M5.Speaker.setVolume(128); // set volume (0-255)

  // Play three notes in sequence (blocking style)
  M5.Speaker.tone(C4, 500); // play C4 for 500 ms
  delay(500); // wait for it to finish
  delay(50); // short gap between notes
  M5.Speaker.tone(E4, 500);
  delay(500);
  delay(50);
```

```
M5.Speaker.tone(G4, 500);  
delay(500);  
delay(50);  
}  
  
void loop() {  
  // nothing here – tones play once in setup()  
}
```

Lab Exercises

Task 1: Button Piano (lab_4_1)

Objective: In this task, you will discover the difference between **blocking** and **non-blocking** playback, and between `tone(freq, duration)` and `tone(freq)` without a duration.

Button A → Do (C4)

Button B → Re (D4)

Button C → Mi (E4)

1.1 Create a new sketch and copy the code below. Save as **lab_4_1**.

```
#include <M5Unified.h>

#define C4 262
#define D4 294
#define E4 330
#define G4 392

int duration = 1000; // play each note for 1 second

void setup() {
  M5.begin();
  M5.Speaker.setVolume(128); // set speaker volume (range 0-255)
  M5.Display.setTextSize(3);
  M5.Display.println("Lab 4 - Task 1");
  M5.Display.println("Button Piano");
  M5.Display.println("A B C");

  // tone(freq) with no duration - play until stop() is called
  M5.Speaker.tone(G4);
}

void loop() {
  M5.update();

  if (M5.BtnA.wasPressed()) { // Button A handler
    M5.Speaker.stop(); // stop G4 or whatever is playing
    M5.Speaker.tone(C4, duration);
    delay(duration); // BLOCKING - no response until done
  }

  if (M5.BtnB.wasPressed()) { // Button B handler
    M5.Speaker.tone(D4, duration); // NON-BLOCKING -- loop() keeps running
  }

  if (M5.BtnC.wasPressed()) { // Button C handler
    // TODO 1.5: always play clean E4 note
    M5.Speaker.tone(E4, duration); // plays on top of D4 if still playing
  }
}
```

- 1.2 Upload the sketch. **Observe:** G4 plays immediately when the board starts — no button needed. This is `tone(G4)` with no duration. Now press Button A: G4 stops and C4 plays. Why did G4 stop when Button A was pressed?
- 1.3 Press Button A -- you should hear note C4. Press Button B -- you should hear note D4. Press Button C -- you should hear note E4.
- 1.4 Press Button A again. While C4 is still playing, quickly press Button C. **Observe:** Button C does not respond until the C4 note finishes. This happens because `delay(duration)` pauses the entire `loop()` -- while M5Stack is waiting, it cannot check **any** button, including BtnC.
- 1.5 Now press Button B then quickly press Button C while D4 is still playing. **Observe:** Button C responds immediately -- no waiting. But listen carefully -- you will hear D4 and E4 playing on top of each other, which sounds unpleasant. This happens because `M5.Speaker.tone()` runs in the background (non-blocking): `loop()` continues immediately, and a new `tone()` call adds a new sound **on top of** whatever is still playing, instead of replacing it.
- 1.6 **TODO:** Fix the Button C handler so that pressing Button C always plays a clean, single E4 note -- even if BtnB's D4 note is still playing.

Hint: `M5.Speaker.stop()` immediately stops any sound currently playing. Call it right before starting the new tone.

Test: press B, then quickly press C -- you should now hear only E4, with no trace of D4.

- 1.6 **TODO:** Answer question Q1, Q2.

Task 2: Auto Melody (lab_4_2)

Objective: Read and understand pseudo code. Store a sequence of notes in an array and play it automatically using a for loop and tempo calculation.

The following is the pseudo code for a sketch that plays a short melody.

```
DEFINE: (Global Variables)
  note frequencies for octave 4 (C4 to B4)
  MSEC_PER_MIN
  melody[]
  noteValues[] // TODO 2.4
  tempo        // TODO 2.5

SETUP:
  initialize M5Stack and speaker
  call showInfo(tempo)
```

```

LOOP:
  DETECT BtnA pressed:
    FOR each note in melody:
      calculate play time (duration) based on tempo and noteValue
      play current note with calculated play time
      wait for play time to finish      // TODO 2.2
      wait short gap between each note // TODO 2.3

FUNCTION:
  showInfo(currentTempo):
    clear LCD and show title and current tempo

```

Sketch from the pseudo code. Create a new sketch and copy the code below. Save as lab_4_2.

```

#include <M5Unified.h>

// Note frequencies for octave 4
#define C4 262
#define D4 294
#define E4 330
#define F4 349
#define G4 392
#define A4 440
#define B4 494

#define MSEC_PER_MIN 60000
#define DEFAULT_TEMPO 120

// Melody: Old MacDonald Had a Farm (E I E I O)
int melody[] = { G4, G4, G4, D4, E4, E4, D4, B4, B4, A4, A4, G4 };
int noteValues[] = { 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1 };
// note value 1 = one beat, 2 = two beats (held longer)

// Automatically count the number of notes -- no need to change this
int numNotes = sizeof(melody) / sizeof(melody[0]);
int tempo = DEFAULT_TEMPO; // BPM - TODO 2.5: try changing this value

void setup() {
  // initialize M5Stack and speaker
  M5.begin();
  M5.Speaker.setVolume(128);
  M5.Display.setTextSize(3);
  showInfo(tempo);
}

void loop() {
  M5.update();
  if (M5.BtnA.wasPressed()) {
    for (int i = 0; i < numNotes; i++) {
      int playTime = noteValues[i] * (MSEC_PER_MIN / tempo);
      M5.Speaker.tone(melody[i], playTime);
      // TODO 2.2: wait for playTime to finish
      // TODO 2.3: wait a short gap between notes
    }
  }
}

// more on next page

```

```
void showInfo(int currentTempo) {
  M5.Display.clear();
  M5.Display.println("Lab 4 - Task 2");
  M5.Display.println("Auto Melody");
  M5.Display.print("BPM: ");
  M5.Display.println(currentTempo);
  M5.Display.println("Press BtnA");
}
```

- 2.1 Upload the sketch. Press Button A. **Observe:** you can barely hear anything or only the last note plays. Why does this happen?
- 2.2 **TODO:** Add `delay(playTime);` after `tone()` inside the `for` loop. Upload and press Button A again – you should now hear each note in sequence.
- 2.3 **TODO:** After the `delay(playTime)` line, add `delay(50);` Listen carefully -- the notes should now sound more separated and distinct.
- 2.4 **TODO:** Replace all values in `noteValues[]` with `{1, 1, 1, 1, 1, 1, 2, 1, 1, 1, 1, 2}`. Upload and listen. Which notes are held longer now? How does it change the melody?
- 2.5 **TODO:** Try changing the `tempo` variable to `60`, `120`, and `240`. What do you observe?
- 2.6 **TODO:** Answer question Q3.

Task 3: Speed Control (lab_4_3)

Objective: Extract playback code into a function and use button to change the melody tempo at runtime.

Button A – play melody at default speed (120 BPM)

Button B – play melody at 0.5x of current speed

Button C – play melody at 2x of current speed

Pseudo code

```
DEFINE: (Global Variables)
  (same as Task 2)
  DEFAULT_TEMPO
  isPlaying = false

SETUP:
  initialize M5Stack and speaker
  call showInfo(tempo)

LOOP:
  Detect button A press:
    Reset tempo to default
    Play melody
    Show info
  Detect Button B press:
    Halve the tempo
    Play melody
    Show info
  Detect Button C press:
```

```

Double the tempo
Play melody
Show info

```

FUNCTION:

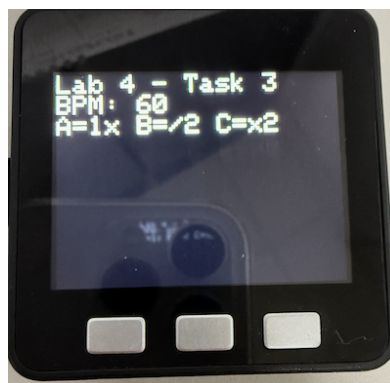
```

playMelody(currentTempo):
    // TODO 3.1: move the for loop from Task 2 here
    calcDuration(noteValue, tempo):
        return play time in milliseconds

showInfo(currentTempo):
    clear LCD and show title, current tempo, and play/stop status

```

- 3.1 **TODO:** Move the `for` loop from Task 2 into the `playMelody(int currentTempo)` function. Call `playMelody(tempo)` from Button A. Verify the melody plays identically to Task 2.
 - 3.2 **TODO:** Add a Button B handler: `tempo = tempo / 2`, then call `showInfo(tempo)` and `playMelody(tempo)`. Press Button B, observe the new BPM on the LCD, then press Button A to compare speeds.
- After you add this, you may notice that the text on the LCD shifts down each time `showInfo()` is called. Find a way to reset the cursor position after `clear()` so the layout stays fixed.
- 3.3 **TODO:** Add a Button C handler: `tempo = tempo * 2`, then call `showInfo(tempo)` and `playMelody(tempo)`.
 - 3.4 **TODO:** Modify Button A so it resets tempo back to `DEFAULT_TEMPO` before calling `showInfo()` and `playMelody()`. This makes Button A, a "normal speed" reset.
- LCD should show the **current speed** when playing melody as shown in pictures below.



- 3.5 **TODO:** Answer question Q4.

Task 4: Play/Pause (lab_4_4)

Objective: Control melody play/pause with Button A (play) and B (pause).

How it works: Instead of a `for` loop that runs to completion, we store the current position in the melody as a global variable `currentIndex`. Each time through `loop()`, if playback is active, we **play one note, advance** `currentIndex`, then return to the top of `loop()` — where buttons are

checked again. This is what makes pause work mid-melody.

LOOP:

```

detect button A pressed
  state is playing
  show current info to LCD

detect button B pressed
  state is pause
  stop the sound immediately
  show current info to LCD

// PLAYBACK LOGIC:
IF in playing state
  get currentNote from melody and currentIndex
  get noteValue from noteValues and currentIndex
  calculate playTime from noteValue and tempo
  play currentNote for playTime
  wait playTime
  wait 50 ms gap
  advance to the next note,
  loop back to the beginning when reaching end of melody
END IF

IF in pause state
  stop the sound immediately
END IF

```

```

#include <M5Unified.h>

#define C4 262
#define D4 294
#define E4 330
#define F4 349
#define G4 392
#define A4 440
#define B4 494

#define MSEC_PER_MIN 60000
#define DEFAULT_TEMPO 120

int melody[]      = { G4, G4, G4, D4, E4, E4, D4, B4, B4, A4, A4, G4 };
int noteValues[] = { 1, 1, 1, 1, 1, 1, 2, 1, 1, 1, 1, 2 };
int numNotes = sizeof(melody) / sizeof(melody[0]);

int tempo = DEFAULT_TEMPO;
int currentIndex = 0;      // which note to play next
bool isPlaying = false; // true = playing, false = paused

void setup() {
  M5.begin();
  M5.Speaker.setVolume(128);
  Serial.begin(115200);
}

```

```

    M5.Display.setTextSize(3);
    showInfo(isPlaying);
}

void loop() {
    M5.update();

    if (M5.BtnA.wasPressed()) {
        isPlaying = true;
        showInfo(isPlaying);
    }

    // TODO 4.2: detect Button B pressed
    //   set isPlaying to false
    //   call M5.Speaker.stop() to halt sound immediately
    //   call showInfo(isPlaying)

    // TODO 4.3: log current state to Serial Monitor

    // TODO 4.1: if isPlaying is true:
    //   get currentNote   = melody[currentIndex]
    //   get noteValue     = noteValues[currentIndex]
    //   calculate playTime from noteValue and tempo
    //   play currentNote for playTime
    //   wait playTime
    //   wait 50 ms gap
    //   advance currentIndex by 1
    //   if currentIndex has reached numNotes, reset it to 0
}

void showInfo(bool currentIsPlaying) {
    M5.Display.clear();
    M5.Display.setCursor(0, 0);
    M5.Display.println("Lab 4 - Task 4");
    // TODO 4.4: if currentIsPlaying, show "B to pause"
    //           else show "A to play"
}

```

- 4.1 **TODO:** Add the playback block inside `loop()` as described in pseudo code and comments. After completing this, press Button A and observe that the melody loops continuously.
- 4.2 **TODO:** Add the Button B pause handler. Press Button B while the melody is playing — you may need to hold it briefly if the program is inside a `delay()`. After pausing, press Button A to resume from where it left off.
- 4.3 **TODO:** Add a `Serial.println()` statement that logs the current state (playing or paused) and `currentIndex` each time the state changes. Open the Serial Monitor and verify.
- 4.4 **TODO:** Complete `showInfo()` so it shows "B to pause" when playing and "A to play" when paused.

Hint: Follow the provided pseudocode for `loop()`.

```

LOOP:
    detect button A pressed
    state is playing

```

```

    show current info to LCD

detect button B pressed
    state is pause
    stop the sound immediately
    show current info to LCD

// PLAYBACK LOGIC:
IF in playing state
    get currentNote from melody and currentIndex
    get noteValue from noteValues and currentIndex
    calculate playTime from noteValue and tempo
    play currentNote for playTime
    wait playTime
    wait 50 ms gap
    advance to the next note,
        loop back to the beginning when reaching end of melody
END IF

```

4.5 **TODO:** Answer Question Q5.

Task 5: Interactive Melody Player (lab_4_5)

Objectives: Write a complete sketch from pseudo-code. Control note playback manually – press to play, release to advance.

- Button C – pressed: start playing the current note
- Button C – released: stop the speaker and advance to next note. Wrap back to note 0 after the last note.

LCD: show the current note number (index) at all times.

Serial Monitor: log "Playing note <index>" when a note starts, and "Stopped. Next note: <index>" when it stops.

Modify the code to create an interactive melody player as the duration of each note is controlled by the user (melody play while the button is held down):

- When you **press and hold Button A**, the current note of the melody should begin to play.
- **Releasing Button A** should immediately stop the sound.
- The melody should only advance to the next note after the button is released and then pressed again.
- When the last note of the melody is played, the next press should restart the melody from the first note.
- Log all button press and release events to the **Serial Monitor**.

DEFINE: (Global Variables)

```
Note frequencies in octave 4
melody[]
numNotes
currentIndex = 0
```

SETUP:

```
Initialize M5Stack and Speaker
Initialize Serial
Show currentIndex on LCD
```

LOOP:

```
Detect a single Button C pressed
  play current note
  LOG "Playing note <currentIndex>" to Serial Monitor

Detect button released
  stop sound
  advance to next note in melody (similar to Task 4)
  show updated currentIndex on LCD
  LOG "Stopped. Next note: <currentIndex>" to Serial Monitor
```

Note: Use `tone(freq)` without duration - the note plays for as long as the button is held. You do not need `noteValues[]` or `tempo` in this task.

TODO: Answer question Q6.

Lab 4: Speaker and Tones

Section: _____ Date: _____

Members

Name: _____ Student ID: _____

Name: _____ Student ID: _____

Name: _____ Student ID: _____

Task		Graded by
1	Button Piano Press B, then quickly press C -- you should now hear only E4, with no trace of D4	
2	Auto Melody Melody plays with correct timing, tempo change observed	
3	Speed Control Button A – reset to DEFAULT TEMPO Button B – halves tempo Button C – doubles tempo LCD show current BPM	
4	Play/Pause Button A – starts/resumes playback Button B – pauses immediately Melody resumes from paused position LCD shows current state Serial Monitor logs events	
5	Interactive Player Button C press – plays current note Button C release – advances to next note LCD and Serial log updated correctly	

6. Answer Questions (in next page)

Q1. In task 1.1, why could Button C not respond while Button A's tone was playing?

Q2. In task 1.2, Button C did respond – but the sound was strange. Explain why two tones overlapped, and how you fixed it in task 1.3.

Q3. In task 2.1 (before adding `delay()`), only the last note could be heard. Use your understanding of `tone()` to explain why.

Q4. In task 3, what happened to the LCD text position each time `showInfo()` was called? What did you add to fix it?

Q5. Explain the key difference between for loop approach in Tasks 2-3 and the `currentIndex` approach in Task 4. Why does the `currentIndex` approach allow pause to work?

Q6. In Task 5, you used `wasPressed()` and `wasReleased()` instead of `isPressed()`. Why is `wasPressed()` the correct choice when the button triggers `tone()`?

7. Individual reflection. Submit as assignment in **myCourseVille**

R1. In Tasks 2 and 3, the melody played from start to finish before your program could respond to any button press. In Task 4, you restructured the code so the program could check buttons between every note. Describe a real-world device (not a music player) where this kind of "check in between every step" behavior would be important. What could go wrong if the device ignored input while busy?

R2. In Task 1, you discovered that calling `tone()` repeatedly without `stop()` caused unexpected sounds. Why do you think the hardware behaved that way, and what does it tell you about the relationship between the code you write and what actually happens inside the device?

R3. Across Tasks 2 through 5, you controlled the same melody in four different ways — fixed playback, speed control, play/pause, and manual note-by-note. If you were designing a simple music toy for a young child, which approach would you choose and why? What trade-offs did you consider?